



SISTEMAS DISTRIBUIDOS DE PROCESAMIENTO DE DATOS II
GRADO EN CIENCIA E INGENIERÍA DE DATOS
EIF + ETSII

PRÁCTICA 2 - SPARK STRUCTURED STREAMING

Procesamiento de flujos de datos estructurados

Autor: Felipe Ortega

24 DE ABRIL DE 2026

 **Profesor:** Felipe Ortega.

 **Versión:** 1.1.

 **Lenguaje de programación:** Python.

 **Entorno de trabajo:** Se recomienda como IDE Microsoft VS Code.

 **Fecha tope de entrega:** Viernes, 8 de mayo de 2026.

 **Envío:** Fichero PDF que contenga las capturas de pantalla solicitadas, así como las respuestas razonadas a las preguntas formuladas en este enunciado; se realizará a través del espacio de entrega habilitado en Aula Virtual. **Un solo envío por cada grupo de trabajo.**

1 Enunciado

El objetivo de esta práctica es familiarizarse con los componentes que proporciona Apache Spark para procesamiento de flujos de datos estructurados, relacionándolos con los conceptos trabajados en la Fase 1 de esta práctica.

- Carga de datos a una cola (*topic*) de Apache Kafka. Comprobación de corrección de datos cargados mediante un cliente consumidor (Python).
- Lectura del flujo de datos desde el *topic* de Apache Kafka en un programa de Apache Spark (PySpark), enlazando las bibliotecas adicionales apropiadas.
- Realización de consultas básicas sobre el flujo de datos estructurado y presentación de resultados.

Esta fase de la Práctica 2 consta de dos apartados:

1. Carga de datos de ejemplo a un *topic* de un *broker* Apache Kafka y comprobación de funcionamiento correcto del proceso de lectura de datos.
2. Comprensión del procedimiento que sigue Apache Spark para leer un flujo de datos estructurados de ese *topic* del *broker* Kafka, así como la realización de consultas básicas usando la API programática de Structured Streaming.

Se recuerda que está disponible una guía básica de programación para Spark Structured Streaming: <https://spark.apache.org/docs/4.1.1/structured-streaming-programming-guide.html>.

También existe otra guía muy básica que explica algunos aspectos para integrar un cliente de Apache Kafka en nuestras aplicaciones con Spark Structured Streaming: <https://spark.apache.org/docs/latest/structured-streaming-kafka-integration.html>.

Advertencia sobre las guías

IMPORTANTE: Estos documentos son guías de referencias. Se recomienda seguir escrupulosamente los pasos descritos en este enunciado para la realización de cada apartado. Las guías pueden no contener todos los detalles necesarios para hacer funcionar las aplicaciones de ejemplo.

1.1 Planteamiento en cada grupo

Esta fase de la práctica es idéntica para todos los grupos de trabajo. Los conceptos adquiridos aquí se aplicarán en la defensa final para cerrar el proyecto de programación, empleando en ese caso los datos asignados a cada grupo.

Se deben descargar los ficheros de código fuente disponibles en el apartado **Materiales** → **Spark Structured Streaming + Kafka**, del Tema 5 en Aula Virtual.

2 Carga de datos en Apache Kafka

Puntuación: 2 puntos

En este apartado preparamos y cargamos los datos de ejemplo en una cola (*topic*) de Apache Kafka, disponible en el mismo servidor empleado para la Fase 1 de la práctica.

Vamos a emplear los ficheros `kafka-producer-confluent.py` y `kafka-consumer-confluent.py`. Estos dos ficheros se deben ejecutar en un terminal (independiente o dentro de VS code) que tenga activado el entorno virtual `kafka` utilizado en la fase anterior de la práctica. Este entorno tiene instalado el paquete `confluent_kafka` que necesitan ambos clientes para funcionar.

1. Edita el fichero `kafka-producer-confluent.py` para que el cliente productor se conecte a la máquina y puerto correspondiente del servicio de Kafka para vuestro grupo (igual que en la fase anterior). Ejecuta varias veces el fichero para producir diferentes tandas de líneas de productos aleatorias que se cargarán en el *topic* `purchases`.
2. Ahora edita el fichero `kafka-consumer-confluent.py` para que se conecte a la máquina y puerto correctos del servicio de Kafka (igual que en la fase anterior). Ejecuta el fichero y comprueba que se consumen sin errores las claves y valores insertados en el paso anterior del *topic* `purchases`.

i Entregable 1: capturas de pantalla de la salida de *scripts* para Kafka

Incluye capturas de pantalla de la salida de la ejecución de los dos *scripts* anteriores, para demostrar que has seguido los pasos correctamente. **(2 puntos)**

3 Kafka + Spark Structured Streaming

3.1 Instalación de PySpark

Vamos a ejecutar PySpark en modo local, aprovechando las características multi-core de los PCs del laboratorio (o de vuestros propios equipos).



Advertencia: Ubicación del entorno virtual en PC Labs EIF

El paquete pyspark que instalamos en este entorno ocupa mucho espacio en disco.

Por favor, en caso de utilizar los PCs de Labs EIF os rogamos que **creéis los entornos virtuales** para este apartado **en el directorio /var/tmp**. Agradecemos de antemano vuestra colaboración.

Los pasos para la instalación son:

- Crea un entorno virtual con Astral uv, con identificador pyspark-411 y que utilice Python v3.11 o superior.
- Dentro del directorio del entorno virtual que acabas de crear, instala la versión 4.1.1 del paquete PySpark.

```
$ uv pip install pyspark[connect]==4.1.1
```

- Verifica que en subdirectorio .venv/bin han aparecido los programas binarios de Spark tras la instalación (spark-shell, spark-submit, etc.).

3.2 Consumidor Kafka desde Spark Structured Streaming

Puntuación: 8 puntos

Activa el entorno virtual pyspark-411.

A continuación, edita el fichero struct_kafka_consumer_local.py para que el flujo de datos de entrada de Kafka se conecte a la máquina y puerto del *broker* Kafka que correspondan a vuestro grupo.

Ahora, ejecuta el fichero y comprueba los resultados de salida. La ejecución debería producirse sin errores.

i Entregable 2: captura de pantalla salida del *script* de Spark

Incluye capturas de pantalla de la salida del *script* anterior que muestren:

- Los mensajes de log que demuestran que se han cargado las bibliotecas de dependencias adecuadas para construir el cliente de lectura de datos desde Kafka.
- Una de las tablas de salida que imprime el programa como resultado.

(2 puntos)

3.2.1 Preguntas

1. ¿Por que pensáis que no se obtiene ningún valor en la primera iteración que imprime por la terminal el resultado de la consulta a la tabla `raw_data` ? (1 punto)
2. ¿Qué función tiene la siguiente línea de código para configurar el *input stream* de Kafka? (1 punto)

```
.option("startingOffsets", "earliest")
```

3. Busca información en la guía de programación de Spark Structured Streaming y las referencias de bibliografía para explicar qué tipo de *stream* de salida crea el siguiente fragmento de código. ¿Por qué podemos hacer consultas directamente sobre dicho *stream*, como si fuese una tabla de datos ordinaria? (2 puntos)

```
describe_query = (input_data.writeStream.queryName("raw_data")  
                  .format("memory").outputMode("append")  
                  .start())
```

4. Implementa dos consultas sobre los datos obtenidos del *stream* consumido desde la cola de Kafka, utilizando dos modos de salida diferentes de Spark Structured Streaming y volcando la salida a un fichero de texto con el nombre `salida[N].txt`, donde [N] se sustituye por el identificador de la salida correspondiente.

i Entregable 3: notebook o *script* de Python con consultas

Se debe entregar el notebook o script Python completo utilizado para obtener los resultados. (4 puntos)

4 Criterios de evaluación

Se trata de una práctica para comprender y consolidar conceptos básicos sobre consumo y procesamiento de flujos de datos estructurados.

Se valoran de acuerdo con la puntuación de cada apartado los siguientes ítems de calificación.

- Corrección de las respuestas para las preguntas de cada apartado.
- Capturas de pantalla para demostrar la ejecución.
- Corrección y ejecución del código para implementar las dos consultas solicitadas con Spark Structured Streaming.

